lecture 8
AVL Tree

ensure $\leq h$ nodes $\rightarrow$ $\leq \log(n)$ leaves

Recall BST
- can perform binary search on the tree
- at the best case in a complete tree, halve the search space at each level
- you lose the benefits if the tree is not balanced

AVL Tree
- retain a balanced tree

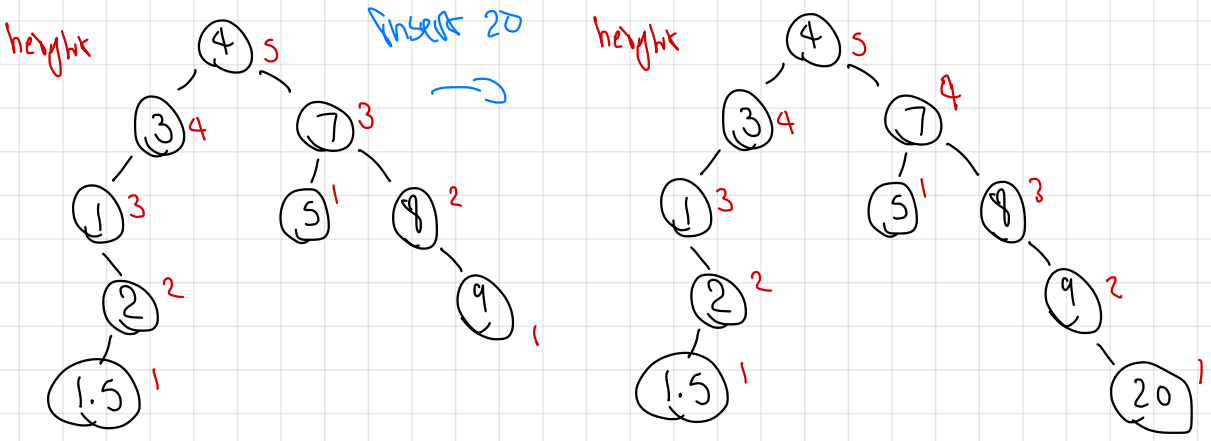
Height
- the maximum length to a leaf out of all children

Conventions

| | empty | 1 node tree |
|---|---|---|
| 0 | 0 | 1 |
| 1 | -1 | 0 |

---

Standard BST — height = 1 + max({height of all children})
- insert 20



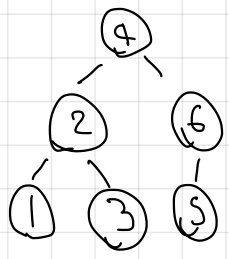

---

can have a height field in BST nodes
just need to compare with existing height after insertion

---

Recall heaps were always complete since the insertion and deletion maintained completeness

Complete BST with values [1,2,3,4,5,6]



only 1, since there is only 1 structure and only 1 ordering for this structure

there is a tradeoff between

- simplicity of structure
- difficulty of update
- difficulty of searches

if sorted
- need to maintain order
- binary search

Decision: it is too hard to maintain a complete BST

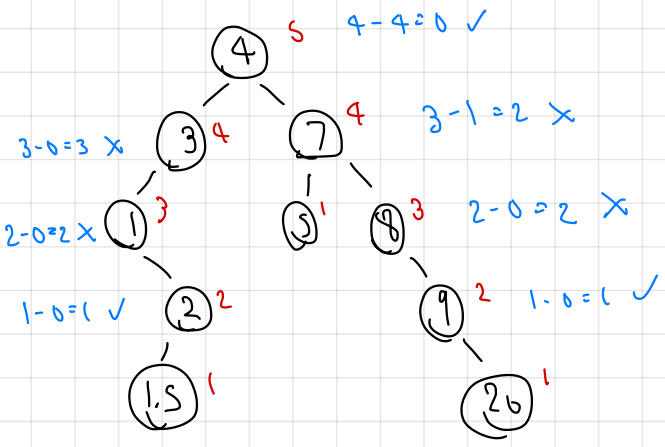
Instead of completeness, use AVL balanced.

Def: A binary tree is AVL balanced if at every node $n$,

$$|height(n.left) - height(n.right)| \leq 1$$

AVL tree is a tree that is AVL Balanced.

---

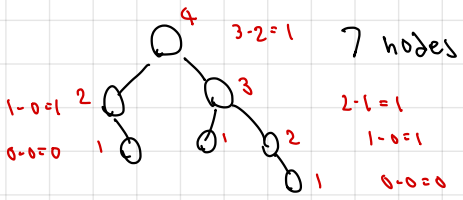



Goal: $\leq n$ nodes means $\leq \log(n)$ levels

Bad: few nodes and large height

---

AVL tree with as few nodes with height 4
- one side: height $h-1$
- other side: height $h-2$




$T_h$ = AVL Tree with height $h$ and the fewest nodes

↑ just to satisfy the AVL balanced property

Get lower bound

$T_h = 1 + T_{h-1} + T_{h-2}$ root +

$T_0$: empty

$T_1$:

$T_2$:

$T_3$:

$T_4$:

---

$C_h$ = # of nodes in $T_h$

Find lower bound

$C_0 = 0$, $C_1 = 1$, $C_h = 1 + C_{h-2} + C_{h-1}$ Fibonacci sequence

smaller

$C'_0 = 0$, $C'_1 = 1$, $C'_h = C'_{h-2} + C'_{h-1}$

smaller

$C''_0 = 0$, $C''_1 = 1$, $C''_h = C''_{h-2} + C''_{h-2} = 2C''_{h-2}$

$C''_h \leq C'_h \leq C_h$ $\geq 2^{\frac{h}{2}}$ 2x nodes per level. $\frac{h}{2}$ levels.

For an AVL tree with height at least $\geq h$, the number of nodes is atleast $n \geq (\sqrt{2})^h$

$(height \geq h) \rightarrow (n \geq \sqrt{2}^h)$

negate both sides and swap

$(n < \sqrt{2}^h) \rightarrow (height < h)$

let $m = (\sqrt{2})^h$

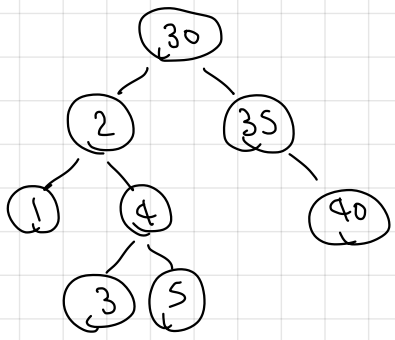
$(n < m) \rightarrow (height < \log_{\sqrt{2}}(m))$

If our tree is AVL balanced and it has at most $n$ nodes, our height will be at most $\log(n)$ nodes

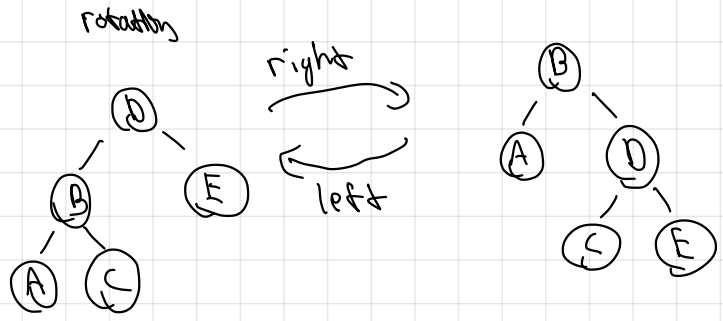
Use rotation to re-establish AVL completeness

make insertion maintain AVL completeness

insert 20 and do rotation



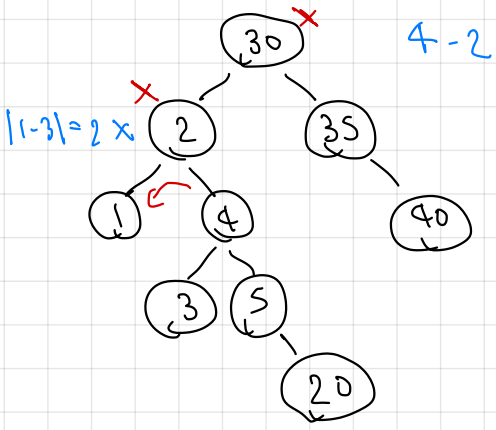

rotation







Insert 20

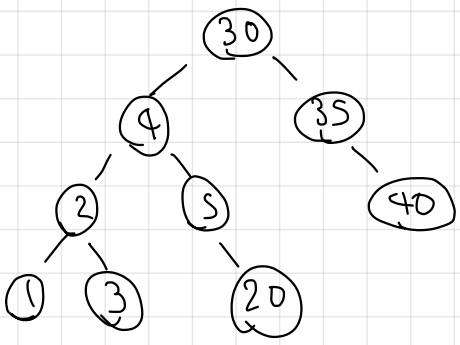



4 - 2 = 2 ✗

|1-3| = 2 ✗

2 unbalanced nodes

rotate left on 2
everything above the rotation stays the same

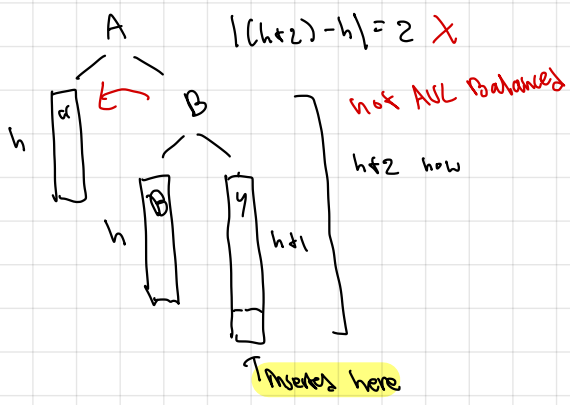



4 becomes new root
2 moves to left
4's left child moves to 4's left right
5 stays as right of 4

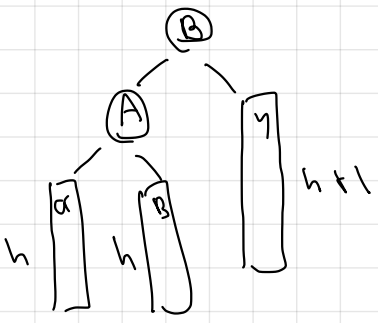
---

there are only 2 cases of an AVL tree to be unbalanced

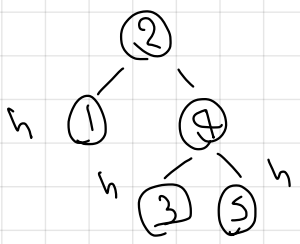
Case 1:
newly inserted node to the right of the right




Solution: rotate A Left
B is now at the top
A is B's left
B's left is now A's right

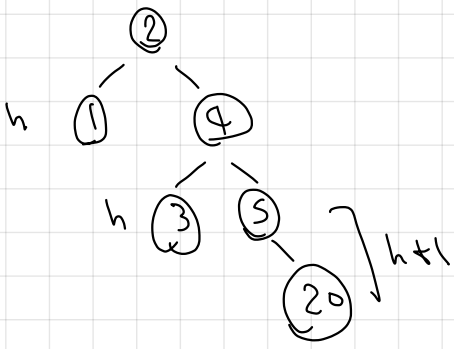



before adding 20

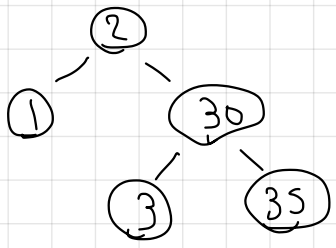



h = 1

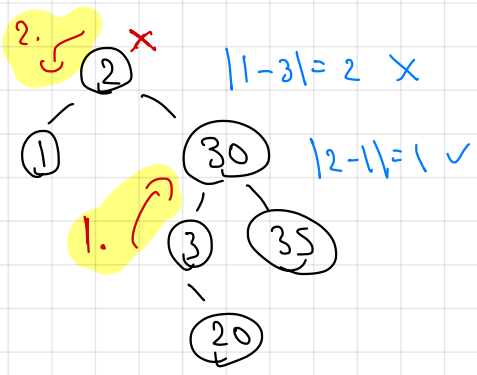
now h+1



Insert 20 and do rotations




Insert 20

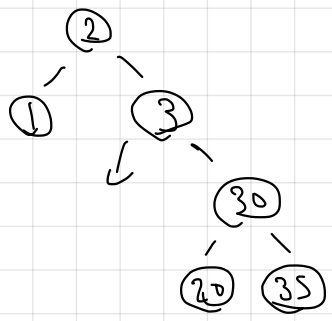



$|1-3| = 2$ ✗

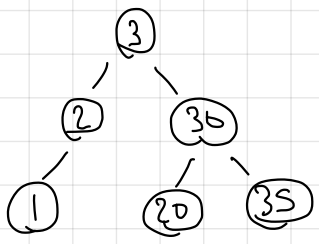
$|2-1| = 1$ ✓

2 rotations
- rotate right on 30
- rotate left on 2

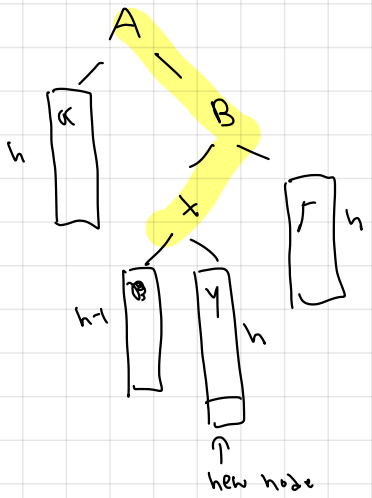
rotate right on 30



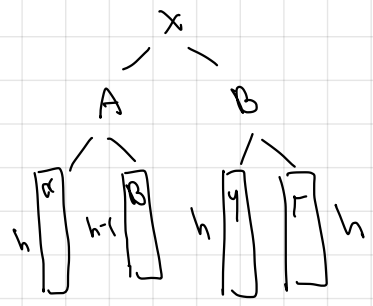

rotate left on 2



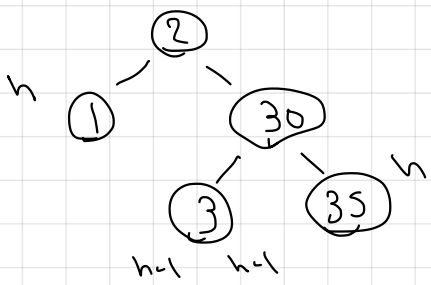

Case 2:



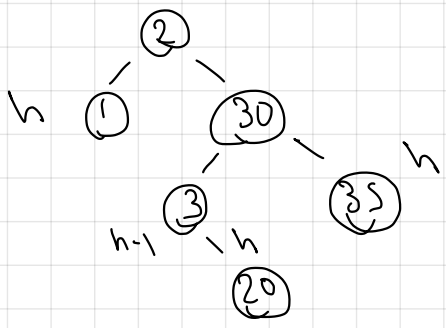

Solution:
1. rotate right at B
2. rotate left at A




before adding 20




now



Strategy  height is saved before and after ancestors are untouched

1. do insertion
2. walk back up ancestors
3. find first unbalanced node
4. if zig-zig, do rotation
5. if zig-zag, do 2 rotations
   - order matters

AVL Insertion $O(\log(n))$ time

Steps
- normal $\log_2(n)$ time
- look at $\log_2(n)$ ancestors
- 1 or 2 rotations at first unbalanced node

Searching will always be $O(\log_2(n))$ time

---

Implementing AVL Trees

Each node will have a height field

After inserting, look at all ancestors of new node and:
- update height
- rebalance

actually, just keep the difference between the left and right heights

# AVL Tree Balance Analysis

AVL tree with $n$ nodes has at most $O(\log(n))$ height

## AVL Balance

- height $h$ must have at least $\sqrt{2}^{h-1}$ nodes
  $= 2^{\frac{h-1}{2}}$

### minimum AVL Tree

$C_h$ = # of nodes

$C_h = 1 + C_{h-1} + C_{h-2}$ $\quad C_0 = 0,\ C_1 = 1$

$C_h > C_{h-1} + C_{h-2}$

$C_h > 2 \cdot C_{h-2}$

2 x more nodes per level
$\frac{h}{2}$ levels

# of nodes $\geq 2^{\frac{h}{2}}$ if $h$ is even

$\geq 2^{\frac{h-1}{2}}$ if $h$ is odd